

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Shaik Basheer	Reg.No.:	192465049
Department:	CSE (Cyber Security)	Date:	01-09-2026

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

*I **Shaik Basheer/192465049**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik Basheer

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

DMA-Based Data Transfer:

Initialize I/O device

Initialize DMA controller

Set source = I/O device

Set destination = Memory

Set data size

Enable DMA

Start data transfer

WHILE transfer is not complete

 CPU performs other tasks

END WHILE

DMA generates interrupt

CPU receives interrupt

IF transfer successful THEN

 Update transfer status

 Process received data

ELSE

 Report transfer error

END IF

Disable DMA STOP

2.Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

Vectored Interrupt Handling System:

START

Initialize interrupt controller

Create interrupt vector table

Assign priority to each device

WHEN interrupt occurs

 Identify interrupting device

 Get interrupt vector

 Check device priority

 IF higher-priority interrupt exists THEN

 Save current CPU state

 Select highest-priority interrupt

 END IF

 Jump to corresponding Service Routine (ISR)

 Execute ISR

 Clear interrupt flag

 Restore CPU state

Return to previous program

STOP

3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Dynamic Selection of Memory-Mapped I/O and I/O-Mapped I/O:

START

Get latency requirement

Get bandwidth requirement

IF latency requirement is HIGH AND bandwidth
requirement is HIGH THEN

 Select Memory-Mapped I/O

ELSE IF bandwidth requirement is LOW AND latency
requirement is LOW THEN

```

        Select I/O-Mapped I/O
    ELSE
        Compare performance of both methods

        IF Memory-Mapped I/O has lower latency THEN
            Select Memory-Mapped I/O
        ELSE
            Select I/O-Mapped I/O
        END IF
    END IF

    Configure selected I/O method
    Perform data transfer

    STOP

```

4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

Bus Arbitration for PCIe, USB and Thunderbolt:

```

START
Initialize PCIe, USB and Thunderbolt devices
Assign priority to each device
Monitor bus requests
WHILE system is running
    Check all bus requests
    IF multiple requests exist THEN
        Compare device priorities
        Select highest-priority request
    ELSE
        Select available request
    END IF
    Grant bus access to selected device
    IF interface = PCIe THEN

```

```

    Perform PCIe data transfer
ELSE IF interface = USB THEN
    Perform USB data transfer
ELSE IF interface = Thunderbolt THEN
    Perform Thunderbolt data transfer
END IF
Release bus
Update device status
END WHILE
STOP

```

5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for highspeed devices.

Hybrid I/O Communication Mechanism:

```

START
Initialize I/O devices
Initialize DMA
Initialize interrupt controller
Initialize polling mechanism
FOR each I/O device
    Check data transfer requirement
    IF large amount of data THEN
        Use DMA for data transfer
        Generate interrupt after completion
    ELSE IF immediate response is required THEN
        Use Interrupt-driven I/O
        Execute appropriate ISR
    ELSE
        Use Polling
        Check device status periodically
    END IF
    Transfer data
    Update device status

```

END FOR

STOP